



ASHESI UNIVERSITY COLLEGE

Anomaly Detection in Rotating Electrical Machinery.

CAPSTONE PROJECT

Nigel Takunda Kadonzvo

B.Sc.in Computer Engineering

2025

Supervisor: Nathan Amanquah

ASHESI UNIVERSITY

Anomaly Detection in Rotating Electrical Machinery.

CAPSTONE PROJECT

Capstone Project submitted to the Department of Engineering, Ashesi University College, in partial fulfilment of the requirements for the award of a Bachelor of Science degree in Computer Engineering.

Nigel Takunda Kadonzvo

2025

DECLARATION

I hereby declare that this capstone is the result of my own original work and that no part of it has been presented for another degree in this university or elsewhere.

Candidate's Signature:.....

Candidate's Name: Nigel Takunda Kadonzvo

Date: 28 April 2025

I hereby declare that the preparation and presentation of this capstone were supervised in accordance with the guidelines on supervision of capstone laid down by Ashesi University College.

Supervisor's Signature:.....

Supervisor's Name: Nathan Amanquah

Date: 28 April 2025

Acknowledgements

First and foremost, I would like to express my heartfelt gratitude to God for His guidance and blessings throughout this journey, which enabled me to complete this project successfully. My sincere thanks go to my supervisor, Dr. Nathan Amanquah, for his invaluable direction, encouragement, and insightful academic advice, which greatly contributed to the success of this work.

I also wish to extend my appreciation to the entire faculty of the Engineering Department for their dedication and for imparting the knowledge and skills that have shaped my academic growth during my time at Ashesi University. I am deeply grateful to my family for their unwavering support, love, and motivation, which have been instrumental in my personal and academic achievements. I want to acknowledge Asher Chakupa, Tendai Machaya and Tinotenda Chagaka for helping me throughout this project.

Lastly, I would like to acknowledge the Mastercard Foundation and the Ashesi Financial Aid Team for their generous sponsorship, which made it possible for me to pursue and complete my four years of study at Ashesi University. Their support has been invaluable to my educational journey.

Abstract

Rotating electrical machines, such as induction motors, are critical assets across various industries, making their reliable operation essential. There are traditional maintenance approaches such as reactive and preventive. They often fall short in optimizing machine uptime and cost-efficiency. This project proposes a portable Internet of Things (IoT)-based system integrated with embedded Machine Learning (ML) to enable real-time anomaly detection in rotating electrical machinery. Collecting multi-modal sensor data (vibration, current, temperature, and thermal imaging) through an ESP32 microcontroller and processing it locally, the system supports condition-based predictive maintenance without heavy reliance on cloud infrastructure. Data acquisition was performed in real-world settings with an induction motor, and in the absence of natural fault data, simulated anomalies were introduced. Multiple ML models, including Random Forests, k-nearest Neighbors, Decision Trees, and Artificial Neural Networks (ANN), were evaluated for their ability to detect faults based on both online datasets and collected vibration data. An ANN deployed on a Raspberry Pi was able to achieve a high classification accuracy of 94%. A web-based dashboard was developed for real-time monitoring and user interaction. This research demonstrates that a lightweight, edge-based predictive maintenance system can significantly enhance the early detection of faults, minimize downtime, and support more innovative asset management in industrial environments, aligning with Industry 4.0 initiatives.

Table of Contents

DECLARATION	i
Acknowledgements.....	ii
Abstract.....	iii
Table of Contents	iv
List of Figures.....	viii
List of Tables.....	ix
CHAPTER 1: INTRODUCTION	1
1.1 Introduction.....	1
1.2 Problem Definition.....	1
1.3 Objectives.....	2
1.4 Project Outcomes	2
1.5 Motivation	3
1.6 Scope of Project	3
1.7 Research Methodology.....	3
CHAPTER 2: LITERATURE REVIEW	5
2.1 Overview of Rotating Electrical Motors	5
2.2 Techniques for Anomaly Detection.....	5
2.3 Condition Monitoring.....	6
2.4 Data Collection and Analysis	6
2.5 Thermal Imaging in Machine Monitoring.....	7

2.6 Machine Learning Algorithms for Anomaly Detection	7
2.7 TinyML Techniques and Frameworks.....	8
2.8 Mean Time Between Failure	8
2.9 Existing Work for Predictive Maintenance	9
2.9.1 Cloud-based IoT Solutions	9
2.10 Challenges	11
2.11 Gaps in Literature.....	11
2.12 Conclusion.....	11
CHAPTER 3: REQUIREMENTS AND DESIGN SPECIFICATIONS.....	13
3.1 Introduction	13
3.2 Design objectives	15
3.3 Requirement Specifications	15
3.3.1 Functional Requirements.....	15
3.4 Criteria for Selecting Modules used for Hardware Design.....	16
3.4.1 Criteria for Selecting a Development Board	16
3.4.2 Criteria for Choosing a Thermal Imaging Sensor	17
3.4.3 Criteria for Choosing Voltage and Current Sensors.	18
3.4.4 Criteria for choosing vibration sensors.....	18
3.5 Motor Selection.....	19
CHAPTER 4: DESIGN AND IMPLEMENTATION	20
4.1 Hardware and Electrical System	20

4.1.1 Circuit Schematic	20
4.1.2 Circuit Calculation.....	22
4.1.3 CAD Design	23
4.1.4 Sensor Placement.....	23
4.2 Fault generation: Inducing Unbalanced Conditions in an Induction Motor	24
4.3 Software System and Implementation	25
4.3.1 Arduino Code.....	25
4.3.2 Database Design	26
4.3.3 ESP32 to Database API	27
4.3.4 Application Dashboard	27
4.3.5 Machine Learning.....	30
CHAPTER 5: RESULTS AND DISCUSSION	34
5.1 System Performance Overview	34
5.2 ESP32, Sensors, Communication, Database and Raspberry Pi	34
5.3 Anomaly Detection.....	34
5.3.1 Running the models on PC	34
5.3.2 Inference on the Raspberry Pi	37
5.3.3 Selection of Machine Learning Model	40
CHAPTER 6: CONCLUSION AND FUTURE WORKS	41
6.1 Conclusion.....	41
6.2 Future Works	41

REFERENCES	43
APPENDIX.....	45

List of Figures

Figure 3. 1: System Design.....	13
Figure 3. 2: Flowchart of the Machine Learning Pipeline	11
Figure 3. 3: System Design of the Machine Learning Deployment	14
Figure 4. 1: Schematic Design for the System.....	20
Figure 4. 2: PCB Design for the System.....	21
Figure 4. 3: Perforated Board Design for the System.....	22
Figure 4. 4: Perforated Board Design for the System.....	23
Figure 4. 5: Sensor placement for signal capture.....	24
Figure 4. 6: Data handling from the sensors to the Web App.	25
Figure 4. 7: The database tables, fields and field names.	26
Figure 4. 8: Users table that stores the login details for the current and new users.....	28
Figure 4. 9: Dashboard: Shows a summary of the data analysis.	28
Figure 4. 10: Dashboard website: Surface temperature of the motor.	29
Figure 4. 11: Dashboard website: Current and Voltage of the motor.....	29
Figure 4. 12: Dashboard website: Vibration analysis page.....	30

List of Tables

Table 3. 1: Selection Criteria for the Microcontroller.....	17
Table 3. 2: Selection Criteria for the Thermal Imaging Camera.....	17
Table 3. 3: Selection Criteria for Current Sensor Selection.....	18
Table 3. 4: Selection Criteria for Vibrator Sensor Selection.....	18
Table 3. 5: Motor Selection Specifications	19
Table 4. 1: Circuit calculation for each component	22

CHAPTER 1: INTRODUCTION

1.1 Introduction.

Rotating electrical machinery such as motors and generators forms the backbone of numerous industries, including manufacturing, transportation, and energy [1]. Failures in these machines often result in significant production downtime, financial loss, and potential safety risks [1]. As a result, predictive maintenance systems are increasingly adopted to preemptively identify faults and reduce unplanned outages. The integration of Internet of Things (IoT) technology with machine learning (ML) introduces new capabilities for real-time monitoring and intelligent fault prediction [2]. This project proposes a portable, IoT-based system combined with embedded ML to detect anomalies in rotating electrical machines. By collecting and analyzing multi-sensor data locally, the system can predict maintenance needs and support more efficient, proactive management of machinery health.

1.2 Problem Definition

Failures in rotating electrical machinery are typically rare but costly when they do occur. Traditional maintenance strategies reactive (post-failure) and preventive (fixed-schedule) fail to adapt to the actual condition of the equipment, often resulting in either premature servicing or unexpected breakdowns [1], [3]. Manual monitoring also introduces risks of oversight and inconsistency. Complicating the issue is the high Mean Time Between Failures (MTBF) for most motors, often exceeding 20,000 hours, which makes collecting real-world failure data within the short duration of a capstone project not feasible [4]. This project addresses the lack of responsive, condition-based maintenance solutions by designing a real-time anomaly detection system that is capable of long-term data collection and local inference using embedded ML. To supplement the scarcity of failure data, the system uses simulated fault scenarios and publicly available datasets to train its ML models.

1.3 Objectives

The primary objectives of this project are as follows:

- To design and implement a portable IoT system capable of continuously monitoring the operational health of rotating electrical machines.
- To collect multi-modal sensor data (vibration, current, and temperature) from machines in real time.
- To apply and compare machine learning algorithms for detecting anomalies based on the collected data.
- To integrate a thermal imaging sensor for identifying heat-related fault patterns.
- To enable real-time, on-device inference using embedded ML frameworks.

1.4 Project Outcomes

Upon the successful completion of this project, the following outcomes are expected:

- A battery-powered, wireless IoT system for collecting and processing real-time machine data.
- A dataset comprising sensor readings under normal and simulated fault conditions.
- A machine learning model trained to detect early signs of mechanical and thermal anomalies.
- Comparative performance metrics for multiple ML algorithms in embedded environments.
- A comprehensive report evaluating the system's practical applications and performance in predictive maintenance.

1.5 Motivation

The motivation for this project arises from the urgent need for efficient, cost-effective predictive maintenance in industrial systems. While motor faults may occur infrequently, even a single unexpected breakdown can halt production lines and incur substantial costs. Traditional strategies fall short of providing tailored, data-driven insights into machinery health. This project embraces Industry 4.0 principles by combining IoT and edge computing to develop a smart, low-power system that operates independently and evolves with time. The platform's portability and ability to run inference without constant internet access make it especially useful in environments with constrained resources.

1.6 Scope of Project

This project is limited to developing and evaluating an IoT-based monitoring and anomaly detection system for rotating electrical machinery, specifically single-phase induction motors. The system includes hardware prototyping, data acquisition, ML model training and deployment, and the use of thermal imaging. It will be deployed and tested in a controlled environment to validate its effectiveness. The focus is not on building a universal model for all machinery types but on establishing a framework that can be generalized and adapted to other equipment in the future.

1.7 Research Methodology

The research methodology for this project will consist of the following steps:

1. **Literature Review:** Analyze current technologies and approaches in IoT-based machine monitoring and anomaly detection.
2. **System Design:** Define the architecture, sensor suite, and communication protocols.
3. **Data Collection:** Acquire operational data from an induction motor in both normal and simulated anomaly states.

4. **Data Analysis:** Preprocess and extract features from sensor data to train ML models.
5. **Implementation and Testing:** Build and deploy the system, integrating hardware and software components.
6. **Evaluation:** Assess model performance, real-time inference capability, and the overall effectiveness of the anomaly detection system.

CHAPTER 2: LITERATURE REVIEW

2.1 Overview of Rotating Electrical Motors

Most of the rotating electrical machines, like induction motors, are very crucial machines that transform electrical energy into mechanical energy for a variety of industrial application. There are several types of motors, such as, induction motors for durability; synchronous motors for precise speed; and brushless DC (BLDC) motors, popularly known for their efficiency [5].

Some of the main components of motors, such as the stator, rotor and bearings, are significant in terms of the motor operation. However, the bearings experience the first signs of failure. Mechanical, electrical and thermal faults are the common problems leading to vibrations, overheating, or less performance [5]. Early anomaly detection in these motors is vital to prevent breakdown and optimize maintenance, thereby improving the reliability and lifetime of the machine.

2.2 Techniques for Anomaly Detection

Anomaly detection helps identify abnormal patterns in machine data, which is crucial for predictive maintenance. Traditional techniques include statistical analysis, signal processing and threshold-based methods, used to detect deviations in motor behaviour [5]. Modern approaches leverage machine learning algorithms like Support Vector Machines (SVM), Decision Trees and Artificial Neural Networks to classify data when labelled information is available [6]. Unsupervised learning methods, such as clustering and Isolation Forest, handle scenarios with limited labelled data. Deep learning techniques like autoencoders are important in identifying anomalies in complex data improving the accuracy and efficiency of fault detection [6], [7].

2.3 Condition Monitoring

Condition monitoring encompasses various techniques, including thermal monitoring, magnetic flux analysis, vibration analysis, partial discharge monitoring, air gap analysis, noise assessment, stator voltage monitoring and current evaluation [8]. Thermal monitoring focuses on tracking temperature changes, which can help detect problems in components like bearings and stators [9]. Vibration analysis is one of the most prevalent methods; it employs accelerometers to identify defects in bearings and other mechanical issues [10]. Traditional monitoring methods can be labour-intensive and may lead to increased operational costs [9]. These approaches may not always give precise predictions, which brings the necessity for IoT-based motor health monitoring systems, combined with machine learning techniques [2].

2.4 Data Collection and Analysis

The quality of the data available impacts the accuracy of the models used in anomaly detection. Most data to be used is related to temperature, vibration, current, voltage and speed; all data are measured with their respective sensors. Most of this data requires preprocessing after collection to be effectively used. Various techniques can be implemented, including removing outlier data, normalization to fit into a spectrum and filtering noise within the selected datasets [11].

Studies have been done to evaluate multi-sensor data fusion on the diagnosis of electrical motors, which show that the integration of various sensors improved the accuracy and precision of predictive models compared to those relying heavily on single sensors [12]. Features in the data were extracted using advanced methods such as Fast Fourier Transform and wavelet analysis, which help transform raw data into machine learning model recognizable features [13].

2.5 Thermal Imaging in Machine Monitoring

Thermal imaging is a technique that uses infrared radiation to create an image of the heat emitted by objects. Every object emits infrared radiation, and the amount of radiation is proportional to its temperature. Thermal imaging can create informative and detailed heat maps containing hot spots and areas of excessive heat buildup by capturing infrared radiation from the emitting machine parts. These temperature anomalies represent the presence of friction, electrical faults, or insufficient cooling, which can be addressed before any failure occurs [14].

Thermal imaging makes non-invasive monitoring possible, so machine inspections are done without interrupting operations. Thermal data image processing techniques are applied by extracting features from the thermal image to identify regular patterns in the machine's thermal behaviour [14]. When linked to machine-learning algorithms, thermal imaging uses more technology to develop anomaly-detection capacity for fault identification [15]. It makes, thus, a more advanced and proactive approach for proactive fault diagnosis, which results in reduced downtime and maintenance cost and enhanced reliability and efficiency of rotating motors.

2.6 Machine Learning Algorithms for Anomaly Detection

Machine learning algorithms are critical in enhancing anomaly detection for rotating electrical machinery by enabling systems to identify abnormal patterns in operational data. Supervised learning techniques, such as Support Vector Machines (SVM), Decision Trees and Neural Networks, are widely used when labelled data is available [6]. These algorithms learn from historical data to distinguish between normal and faulty machine states, making them effective for predictive maintenance. Unsupervised learning algorithms, like k-means Clustering, Gaussian Mixture Models and Isolation Forests, are useful when labelled data is scarce [6]. These techniques focus on identifying outliers in the data that course unexpected patterns, identifying potential anomalies.

2.7 TinyML Techniques and Frameworks

Tiny Machine Learning (TinyML) is the concept of applying machine learning algorithms on microcontrollers other resource-constrained devices. This makes it ideal for real-time anomaly detection in rotating electrical machinery [14]. This approach enables intelligent processing directly at the edge, reducing latency and the need for continuous cloud connectivity. Techniques in TinyML focus on optimizing machine learning models to run efficiently on low-power hardware with limited computational resources, ensuring that they consume minimal memory and energy [15], [16]

Many authors used popular frameworks for implementing TinyML, including TensorFlow Lite for Microcontrollers, Edge Impulse and Arduino's ML libraries. TensorFlow Lite for Microcontrollers provides tools for creating compact models that can be deployed on microcontroller units [16]. Edge Impulse a user-friendly environment for model building and training targeted at embedded devices with real-time applications [17]. ML libraries integrate seamlessly with various microcontroller boards, enabling easy development and deployment of TinyML models. The ML libraries integrate well with various microcontroller boards, making developing and deploying TinyML models easy. These frameworks empower developers to bring machine learning capabilities to small, cost-effective devices, allowing for more responsive and efficient anomaly detection in machine health monitoring systems.

2.8 Mean Time Between Failure

Mean Time Before Failure (MTBF) in induction motor parameter is usually given for higher volumes-industry applications since a failure in such instance may lead to major operational and even financial losses [4]. This mean time before failure is defined as the average time between two unavoidable incidences of failure during normal functioning of a component or system. The MTBF of induction motors varies based on factors such as motor size, voltage,

application and maintenance practices [4]. Figure 2.1 summarizes the findings on the different machine types, their corresponding failure rates, and their Mean Time Before Failure in hours. This information lays a foundation for this research, proving that it is pretty difficult to operate an induction motor until it reaches failure.

Machine type	No of machines in survey	Machine years in survey	No of failures in survey	Failure rate, failures/machine/yr	MTBF, hrs	Source of data
Large steam turbogenerators	Not known	762	24	0.0315	2 73 750	Dickinson, 1974 [10] also IEEE Gold Book [9]
Motors greater than 75 kW generally MV and HV induction motors	6312	41 614	1474	0.0354	2 47 312	Albrecht, 1986 [13]
Wind turbine generators <2 MW	643	5173	710	0.0400	2 19 000	Tavner, van Bussel, Spinato, 2006 [18]
Induction motors 601-15000 V	Not known	4229	171	0.0404	2 16 831	Dickinson, 1974 [10] also IEEE Gold Book [9]
Motors greater than 11 kW generally MV & HV induction motors	2596	25 622	1637	0.0639	1 37 109	Thorsen, 1995 [15]
Motors greater than 150 kW generally MV & HV induction motors	1141	5085	360	0.0707	1 23 735	O'Donnell, 1985 [12] also IEEE Gold Book [9]

Figure 2. 1: A summary of MTBF for different induction motors.

The time needed to cause an induction motor to fail stretched beyond the research's timeline. So, to get faulty/abnormal data, artificial ways of causing anomalies were explored, including supplying imbalanced power to an induction motor, which will cause it to vibrate vigorously.

2.9 Existing Work for Predictive Maintenance

2.9.1 Cloud-based IoT Solutions

Cloud-based IoT solutions have significantly advanced predictive maintenance for electric motors. The solutions allow for real-time remote monitoring. They also allow for smart analytics. In a typical setup, the sensors are attached to the motors, continuously collecting and sending data over the internet to the cloud, which can be stored and analyzed for early signs of

potential faults. Platforms like IBM Maximo Predict use the cloud’s powerful computing capabilities to apply machine learning algorithms.

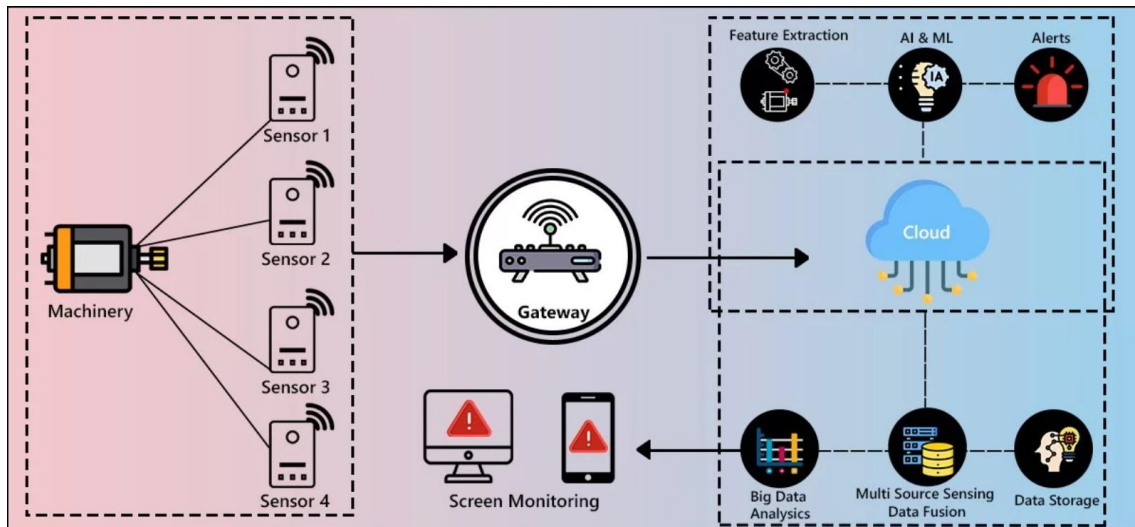


Figure 2. 2: Architecture for Cloud-Based IoT solution.

Figure 2.2 shows a simple architecture for cloud-based solutions and the various components in the system. Such a solution is suitable for big companies that can cover the costs needed to run the cloud services. However, cloud-based systems have challenges. They require stable internet, which can be problematic in remote or harsh industrial environments. There are also concerns around data security, privacy and latency. Because of these limitations, there’s growing interest in hybrid solutions that balance cloud and edge computing.

A predictive maintenance system for motors that collected operational data and transmitted it to a cloud platform for storage and anomaly detection inference [18]. Their solutions demonstrated the use of cloud computing and IoT. However, the project had key limitations. The machine learning models were trained solely on an online dataset. They did not collect real data from the motor they intended to use [18]. This limited the ability to validate the model’s performance under actual operating conditions. In addition, relying entirely on cloud-based inference introduced potential latency and network dependency challenges. Also, the communication was not wireless [18].

2.10 Challenges

The key issue observed in creating models that can be effective in anomaly detection is the lack of availability of labelled data, which has inspired the rise and use of unsupervised models, which do not rely on huge available datasets. The industries that make use of the anomaly detection models need real-time access to data and communication, instead of storing the data on the cloud first. This leads to the adoption and use of embedded machine learning, as this can allow detection at the device level, minimizing latency and other related costs such as communication with the cloud facilities for data processing.

2.11 Gaps in Literature

Despite advances in predictive maintenance systems for motors, there are still gaps. Many solutions depend heavily on cloud-based architectures, which results in issues related to internet dependency, high costs and latency. Additionally, most projects relied solely on online datasets without collecting real operational data. This limits model validation under real-world conditions. There is limited use of edge computing approaches. Edge computing enables faster, more reliable and offline anomaly detection.

2.12 Conclusion

Integrating techniques such as anomaly detection and machine learning into condition monitoring has significantly benefited the reliability and efficiency of rotating electrical machines. Although traditional methods are meaningful, their limitations are chiefly in speed and precision and therefore point toward new IoT-based systems that depend on real-time data analysis [2]. Machine learning algorithms offer promising advancements for predictive maintenance, enabling early fault detection even in resource-constrained environments [7], [15]. However, challenges such as data scarcity and the need for more generalized models

remain, highlighting areas where further research is essential. Addressing these gaps will be crucial to advancing the practical application of these technologies in industrial settings.

CHAPTER 3: REQUIREMENTS AND DESIGN SPECIFICATIONS

3.1 Introduction

The project's primary goal is to build a system that can collect, process and store data (current values, voltage, acceleration from the vibration sensors and temperature variations) while also making it easy to access for monitoring. It also aims to help users find problems and perform maintenance before things break down.

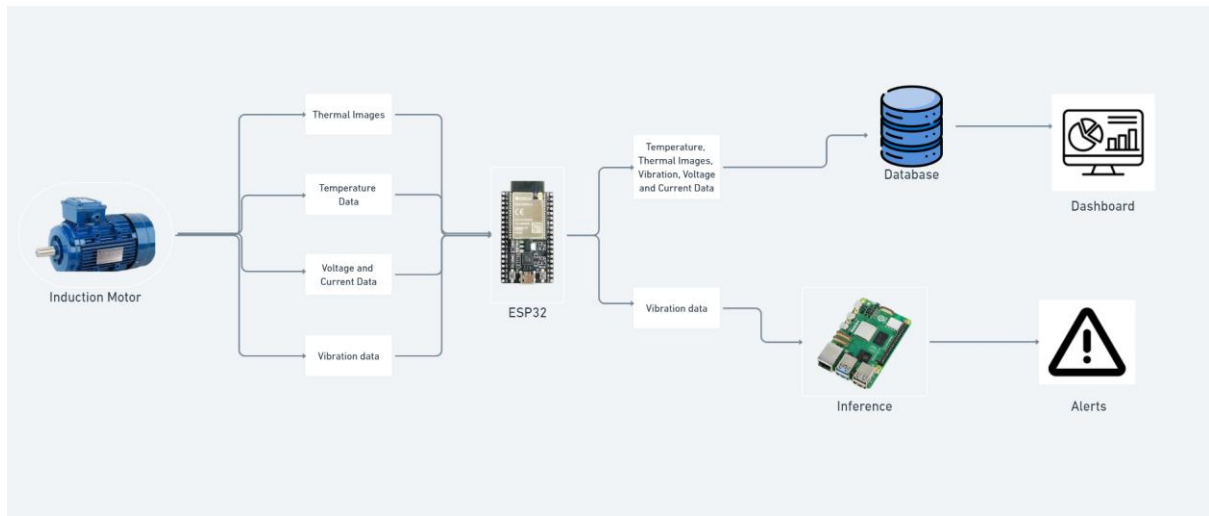


Figure 3. 1: System Architecture

Figure 3.1 illustrates the system's workflow. Vibration, temperature, thermal distribution, voltage and current will be recorded, as these are key parameters for predictive maintenance. Sensors will collect this data, which will be stored in a database and then be used to train the machine learning model when enough data is collected. A model is trained and deployed onto a development board for edge processing. The device will collect and analyze vibration data in real-time. It will be detecting anomalies that indicate the need for early maintenance before issues escalate.

The flow diagram in Figure 3.2 illustrates the machine learning pipeline for anomaly detection in rotating motors. It begins with data collection, followed by data pre-processing to clean and normalize the data.

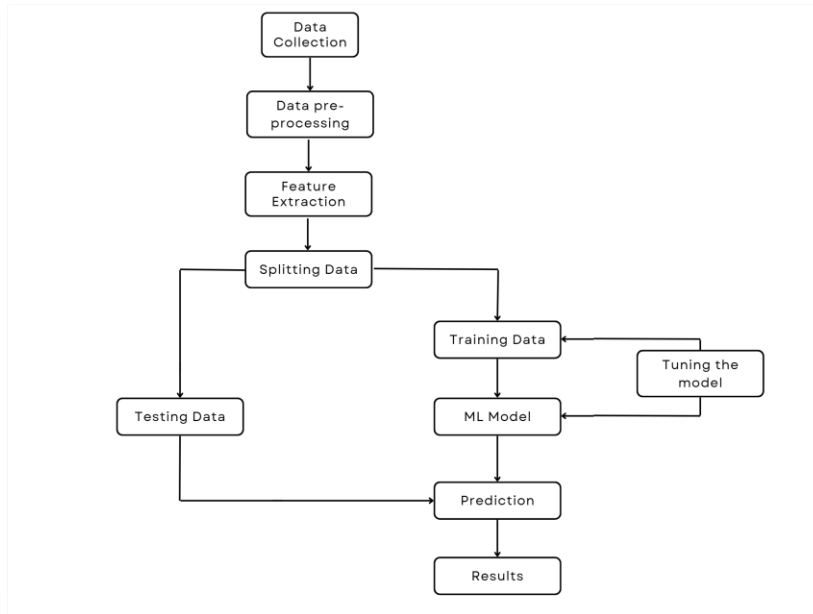


Figure 3. 2: Flowchart of the Machine learning Pipeline

Feature extraction identifies relevant patterns, which are then split into two: training data and testing data. The training data is used to train the ML model by tuning it and optimizing its performance. The trained model makes predictions, and the results are evaluated using the testing data to assess accuracy and effectiveness before using real time data.

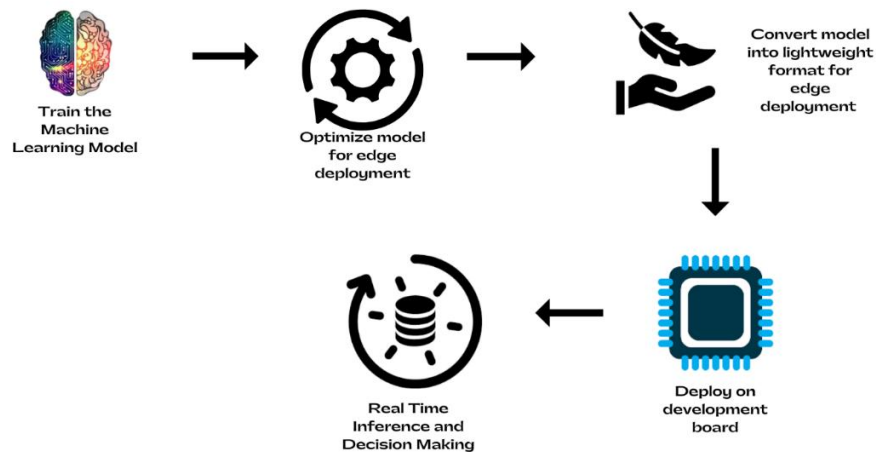


Figure 3. 3: System Design of the Machine Learning Deployment

The flow diagram in Figure 3.3 represents the stages involved in deploying a machine-learning model on an edge device. Starting with the training of a machine-learning model, the next step

is optimizing the model toward edge deployment so as to be efficient. Then, the model is converted into a lightweight version for compatibility with resource-constrained edge devices. It is then deployed onto the development board for independent operations. The last step in the process is where this deployed model facilitates real-time inference and decision-making, assuring fast and efficient process execution on the edge device.

3.2 Design objectives

To design early fault-detecting devices in an induction motor that has the following features:

- A system that gives early warnings when an anomaly is detected (voltage, temperature, vibration, thermal distribution) and should prompt the users for maintenance.
- A system that should record and store data in a MySQL database hosted locally via XAMPP for future model training or reference when needed.

3.3 Requirement Specifications

3.3.1 Functional Requirements

The system aims to enhance motor reliability, minimize downtime and enable predictive maintenance by accurately identifying potential faults in real-time. The following functional requirements outline the key capabilities and features needed to ensure the successful implementation and performance of the system.

- The system must process temperature variations and provide fault detection results in real-time or near real-time.
- The system should be able to capture vibrations and provide fault detection results in real-time or near real-time.
- The system should classify different types of motor faults and provide detailed fault diagnostics.

- The system should generate alerts (e.g., notifications) when potential faults are detected.
- The machine learning model should adapt to the new data over time to improve performance and reduce false positives/negatives.
- The system should include a user-friendly interface for monitoring motor status, viewing fault logs and managing settings.
- The system should incorporate a database or storage system for historical data and fault analysis.
- The systems should minimize power consumption to be feasible for continuous use, for example by using energy-efficient hardware components, optimizing software algorithms to reduce processing time, and implementing sleep modes during idle periods.

3.4 Criteria for Selecting Modules Used for Hardware Design

The selection of hardware components is essential to ensure optimal performance and functionality. Each component was chosen based on specific criteria, including performance, power consumption, cost, and suitability for the application.

3.4.1 Criteria for Selecting a Development Board

The ESP32 was chosen as the baseline due to its balanced performance across key factors. It offers good memory and speed, built-in ML capabilities with vector instructions, low power consumption and strong wireless connectivity (Wi-Fi + Bluetooth). Additionally, it provides a cost-effective solution compared to more powerful but expensive options like the Raspberry Pi 4, making it ideal for embedded AI and IoT applications.

Table 3. 1: Selection Criteria for the Microcontroller

Criteria	ESP32 (Baseline)	Arduino Nano RP2040	STM32 Nucleo	Raspberry Pi 4
Memory	0	-2	1	3
Speed	0	-3	3	1
Power	0	1	-3	-4
Connectivity	0	-1	-2	2
Cost	0	0	-3	-5
Total	0	-5	-6	0

3.4.2 Criteria for Choosing a Thermal Imaging Sensor

The requirements demanded a thermal sensor with high resolution, accuracy, low power consumption, and reasonable frame rate. The sensor should also have a wide range and field of view (FOV) to capture thermal variations effectively. The MLX90640 was chosen due to its 32x24 pixel resolution, high accuracy, and 16Hz frame rate. Although more expensive than the AMG8833 and TPA81, it offers superior range (55° FOV) and thermal sensitivity, making it ideal for detailed thermal monitoring.

Table 3. 2: Selection Criteria for the Thermal Imaging Camera

Criteria	MLX90640 (Baseline)	AMG8833	TPA81
Resolution	0	-2	-3
Accuracy	0	-3	-2
Cost	0	1	2
Power Consumption	0	-2	2
Range	0	-1	-2
Frame Rate	0	-2	-3
Field of View (FOV)	0	1	-2
Thermal Sensitivity	0	-1	-2
Compatibility	0	0	-1
Total	0	-9	-11

3.4.3 Criteria for Choosing Voltage and Current Sensors.

The PZEM-004T was chosen as the baseline current sensor due to its ease of integration, built-in voltage and current measurement capabilities, and reliable performance for general-purpose applications. While it may not offer the high precision of the INA226 or the low cost of the ACS712, it provides a balanced combination of features, including a wide measurement range and digital output, making it a suitable reference point for comparison in typical IoT-based monitoring systems.

Table 3. 3: Selection Criteria for Current Sensor Selection

Criteria	PZEM-004t (Baseline)	INA226	P ACS712
Resolution	0	0	-1
Accuracy	0	1	-1
Cost	0	-1	1
Range	0	0	1
Response Time	0	-1	-2
Output Type	0	0	-1
Total	0	-1	-3

3.4.4 Criteria for choosing vibration sensors

The MPU6050 was chosen as the baseline due to its balanced performance, affordability, and wide compatibility with microcontrollers. While it lacks the high sensitivity and durability of the ADXL1002, it provides better cost efficiency and adequate resolution and range than the LIS3DH, making it a better choice.

Table 3. 4: Selection Criteria for Vibrator Sensor Selection

Criteria	MPU6050 (Baseline)	ADXL1002	LIS3DH
Resolution	0	-2	-1

Accuracy	0	-3	-2
Cost	0	1	2
Power Consumption	0	-2	1
Sensitivity	0	2	-1
Range	0	-1	-2
Durability	0	2	-1
Bandwidth	0	1	-1
Compatibility	0	0	1
Total	0	-2	-4

3.5 Motor Selection

To test the condition monitoring system I had select a suitable motor. The motor that was available and whose data could be collected was an AC motor provided by the school staff.

The table below shows the specifications of the physical motor used for the system.

Table 3. 5: Motor Selection Specifications

Power Requirement	240/415 V
Output Power	175 kW
Stator Voltage	240/415 V, 3-phase
Rotor Voltage	120/208 V, 3-phase
Full-Load speed	1315 r/min
Full-Load Current	0.48 A
Dimensions (HxWxD)	308 x 291 x 440mm
Net Weight	14 kg

CHAPTER 4: DESIGN AND IMPLEMENTATION

This project involves developing a monitoring system for an induction motor using the ESP32, the Raspberry Pi 5 and various sensors to track parameters such as current, voltage, vibration, thermal images and temperature. This chapter highlights the physical and software design and implementation of the project. It explains how the different sub-systems of the project were designed and implemented.

4.1 Hardware and Electrical System

4.1.1 Circuit Schematic

The schematic diagram of the electrical system was made with EasyEDA, as shown in Figure 4.1. One of the aims of the system is reliable power supply. To improve the stability and reliability of the system, external power supplies were designed with rechargeable batteries.

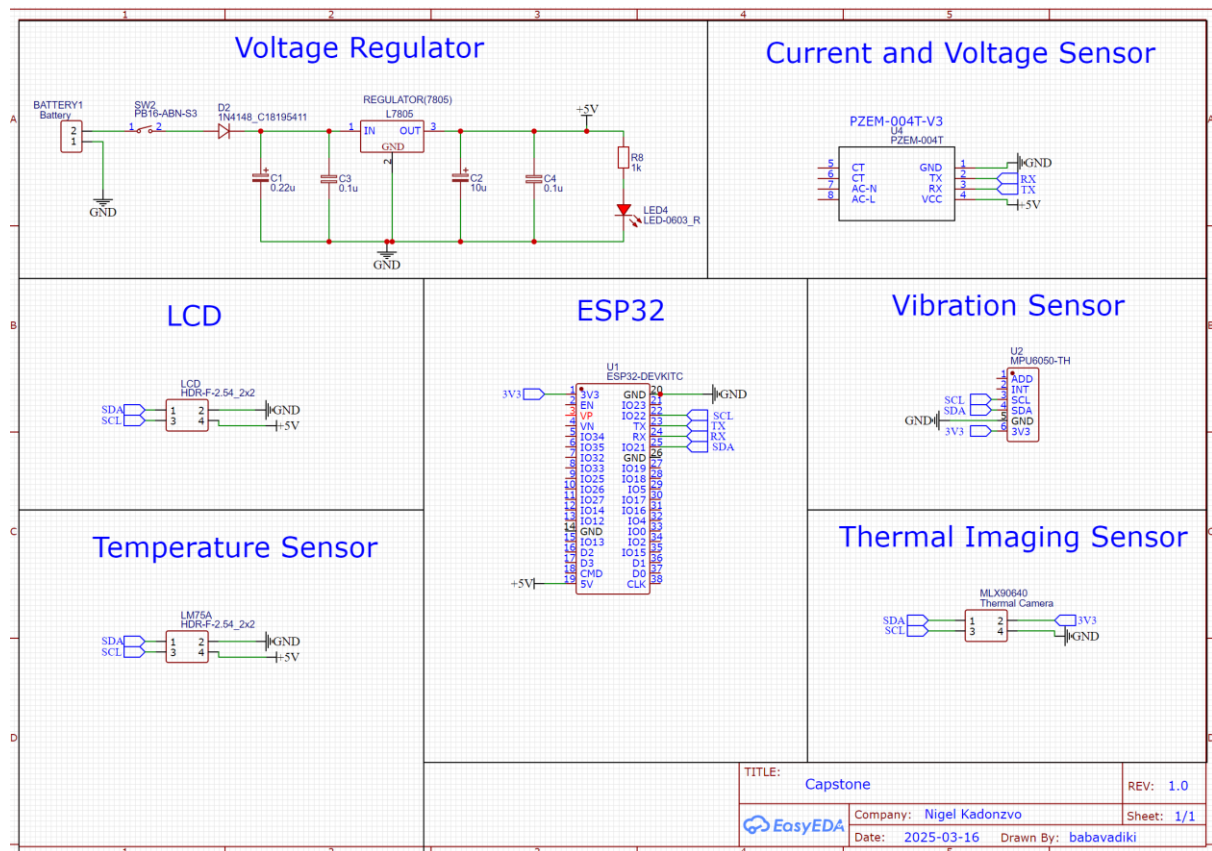


Figure 4. 1: Schematic Design for the System

Considering the voltage sensitivity of the ESP32, additional electrical components, along with the voltage regulator (LM7805), were used in these external designs to stabilize a smooth voltage output of 5.04 volts needed for the microprocessor to operate properly.

The schematic includes the different sensors that are being used to collect data and monitor the condition of the induction motor. The PZEM-004t measures the current and voltage of the induction motor. The LM75A is a temperature sensor that measures the surface temperature of the induction motor as it operates. The MPU6050 is an accelerometer that measures the vibrations produced by the induction motor during operation. It measures both linear and rotational acceleration in the x, y and z planes.

The LCD was used as a display to print sensor readings continuously at intervals of one second. This update rate was chosen to provide near real-time feedback to the user while ensuring the data remained readable and the system performance was not negatively affected by excessive refresh rates. The MLX90640 measures the thermal distribution in the induction motor.

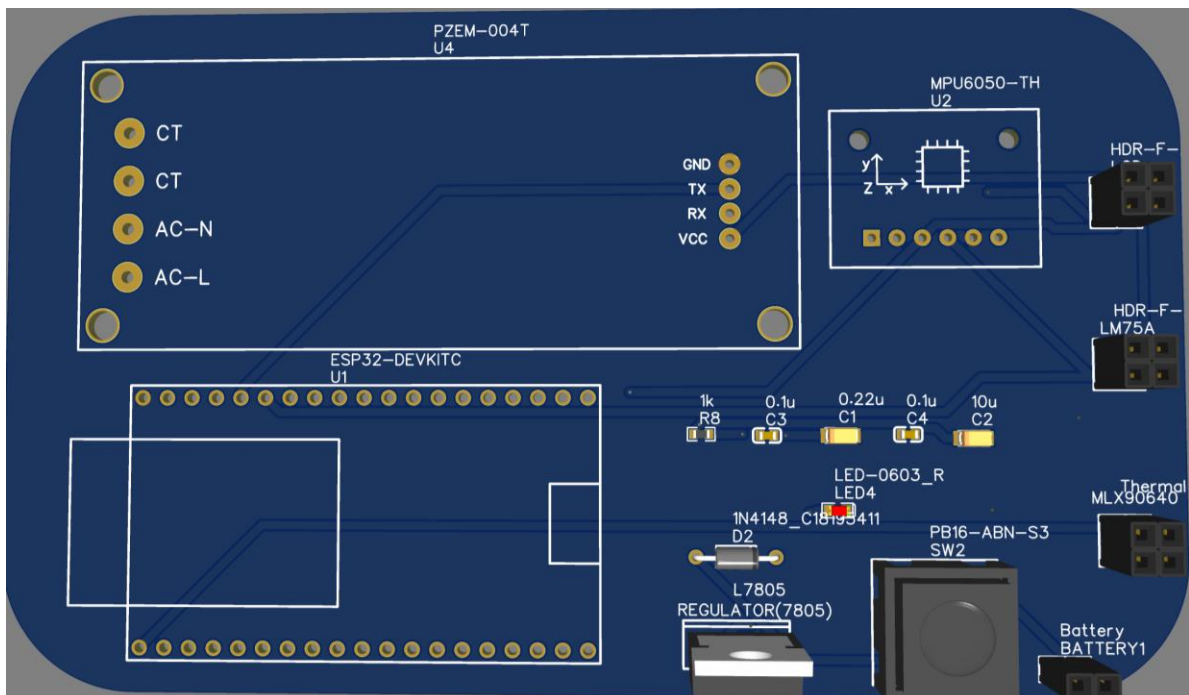


Figure 4. 2: PCB Design for the System

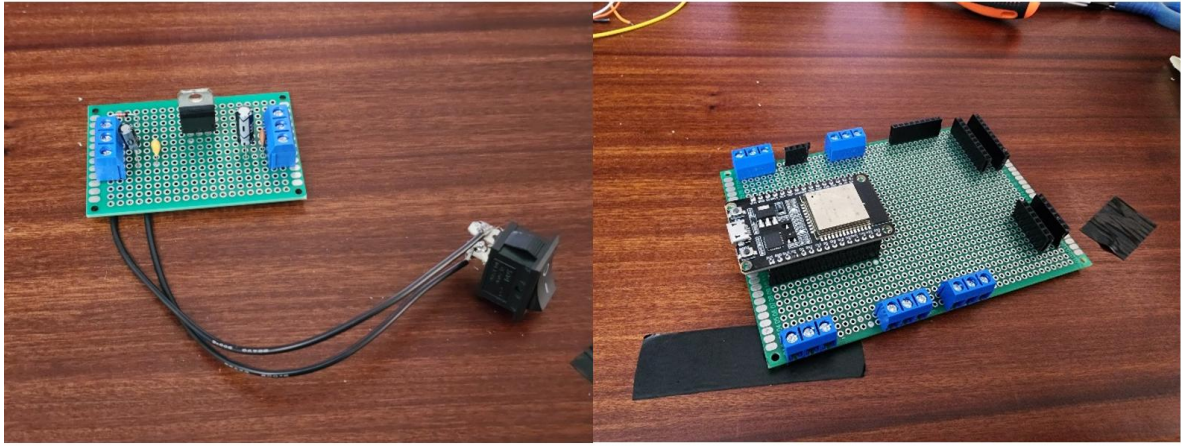


Figure 4. 3:Perforated Board Design for the System

Figure 4.2 shows the PCB design of the electrical system. Instead, the circuit components were mounted on perforated (perf) boards to create a compact and organised layout for the electrical system, as shown in Figure 4.3. The left image shows the power regulation board, which includes a voltage regulator (LM7805) and supporting passive components, designed to provide a stable 5.04-volt output from a higher voltage input source. It also includes a switch used to turn the system on and off. The right image in Figure 4.2 displays the main control board featuring the ESP32 microcontroller, also mounted on a perf board. The wiring connections remain clean and secured using terminal blocks and female header pins. The setup serves to integrate sensors and connected devices via surrounding terminal blocks and headers, providing the basis for reliable prototyping and future enhancements.

4.1.2 Circuit Calculation

Table 4. 1: Circuit calculation for each component

Component	Voltage requirement (V)	Current requirement (A)
ESP32	3.3	(160-260) m
Raspberry Pi 5	5	1.6
MPU6050	3.3-5	4m
LCD	5	1m
Buzzer	1.5 - 12	20m
ACS712	5	10m
LM75A	3.3 - 5	1m

MLX90640	3.3 - 5	23m
-----------------	---------	-----

4.1.3 CAD Design

A CAD design for the casing of the electronic components was created using SOLIDWORKS to ensure an optimal layout and portability for the system. The design provides sufficient space for all components while offering structural integrity. Figure 4.4 shows the CAD model of the casing, which features cutouts and slots for proper component placement. The final casing was fabricated from plexiglass, as shown in Figure 4.4, offering both durability and visibility for the internal components. This plexiglass casing enhances the system's portability while protecting the sensitive electronics.



Figure 4. 4: Perforated Board Design for the System

4.1.4 Sensor Placement

The MPU6050 accelerometer was positioned on top of the induction motor. Black tape was used as a temporary mount. The accelerometer was mounted on the top surface of the motor. The LM75A temperature sensor was placed on the surface of the induction motor. This was to get the accurate change in temperature in the motor as it runs over a period of time. The LM75A sensor was temporarily mounted to the surface using the black tape. The PZEM-004t sensor was connected to the 3-phase power line to measure the stator current.

Figure 4.5 shows the setup for the Raspberry Pi, with an LCD that prints out the state of the motor's vibration. When the vibration is normal, it prints "No Fault"; when it is unusual, it prints "Fault Detected".

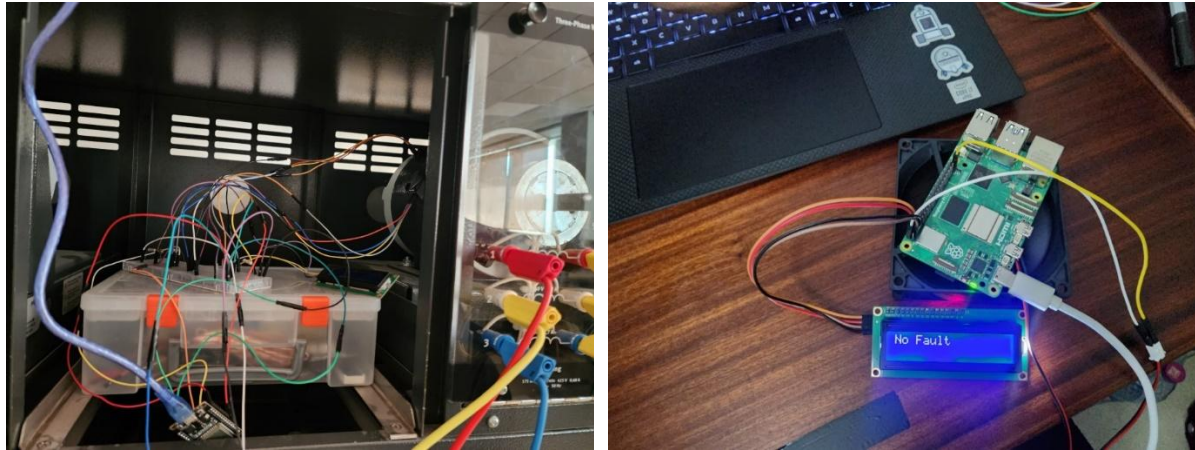


Figure 4. 5: Sensor placement for signal capture

4.2 Fault Generation: Inducing Unbalanced Conditions in an Induction Motor

To simulate abnormal conditions in an induction motor, an unbalanced power supply was intentionally introduced to create artificial faults [19]. This was done by supplying an imbalanced current or voltage to the motor, which is a common issue in industrial settings. Unbalanced loading can cause excessive vibrations and mechanical wear, which can eventually lead to motor failure. Vibration levels were monitored during the fault simulation using accelerometers. These methods helped detect deviations from normal operating conditions [19].

To simulate abnormal conditions in an induction motor, an unbalanced power supply was intentionally introduced to create artificial faults [19]. This was done by supplying an imbalanced current or voltage to the motor, which is a common issue in industrial settings. Unbalanced loading can cause excessive vibrations and mechanical wear, which can eventually lead to motor failure. Vibration levels were monitored during the fault simulation using

accelerometers. These methods helped detect deviations from normal operating conditions [19].

4.3 Software System and Implementation

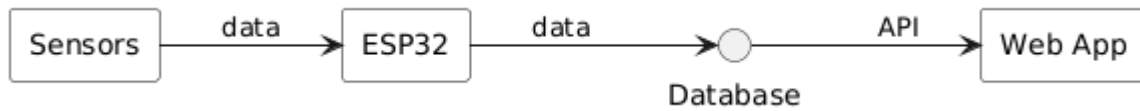


Figure 4. 6: Data handling from the sensors to the Web App.

The data measured by the sensors will be processed in this system, stored in the database, and made easier to access through a web application shown in Figure 4.6. Data will be collected first using a set of sensors, which will be connected to an ESP32 to measure temperature, vibration or other parameters. Then the data is processed and formatted by the ESP32 and sent to the central database through an API service. There are structured tables so that an easy query and retrieval can be done.

The existing API communicates with the web app and the database. Hence, RESTful endpoints secure access to the data in storage. This web application will fetch the data from the API and visualize it to the user, transforming the raw data into interpretable charts and tables. The following sections show the implementation of each of the components of the whole system: the ESP32 code for data collection, the API for handling the data, the database structure, and the web app to enable the users to interact with the data.

4.3.1 Arduino Code

The ESP32 was programmed using the Arduino IDE to monitor voltage, current, temperature, and vibration in real-time. The PZEM-004T V3.0 sensor was used to measure voltage and current through UART communication, providing accurate electrical readings without manual calibration. Temperature data was collected from the LM75A (external temperature via I2C)

and the MPU6050, which also measured motion. The MPU6050 was configured with an $\pm 8G$ accelerometer range and a $\pm 500^\circ/s$ gyroscope range to detect vibrations along the X, Y, and Z axes. Vibration data was sampled at a rate of 100 Hz (every 10 ms) to capture fine-grained motion changes for motor stability analysis. 100 readings for vibration were stored in a batch and then posted to the database.

A 16x2 I2C LCD provided real-time feedback on voltage, current, and temperature values. All sensor data was structured as JSON using the ArduinoJSON library and transmitted to a remote server via HTTP POST over Wi-Fi. The project also included Over-the-Air (OTA) update functionality, allowing wireless firmware updates to the ESP32. This made it easy to deploy code changes without physical access, improving maintainability and flexibility in the field.

4.3.2 Database Design

A MySQL database was created to hold sensor data. Two separate tables were created since the vibration data was collected faster than the other sensor values, like temperature, voltage, and current. The reason was that the changes in vibrations happen very fast. Hence, the system has to capture the values as quickly as possible to capture those changes. Different tables were created for the different loads that will be added to the motor.

capstone users	capstone alerts	capstone vibration_data	capstone othersensors
id : int(11)	id : int(11)	id : int(11)	id : int(11)
name : varchar(100)	type : varchar(50)	accX : float	current : float
email : varchar(100)	message : text	accY : float	voltage : float
phone : varchar(20)	status : varchar(20)	accZ : float	external_temperature : float
password : varchar(255)	sensor_value : float	gyroX : float	mpu_temperature : float
preferences : text	timestamp : timestamp	gyroY : float	timestamp : timestamp
created_at : timestamp	resolved : tinyint(1)	gyroZ : float	dstatus : varchar(50)
		timestamp : timestamp	
		dstatus : varchar(50)	

Figure 4. 7: The database tables, fields and field names.

4.3.3 ESP32 to Database API

A PHP-based API was developed to create a reliable communication between the ESP32 microcontroller and a MySQL database. The API is designed to receive vibration data in batches, formatted as JSON, and transmitted through HTTP POST requests. This batch processing method supports high-frequency data sampling at 100Hz (100 vibration values collected per second). It helps reduce network overhead and ensure timely data logging.

Upon receiving a request, the API connects to the vibration database using specified server credentials. It then validates the presence of a batch array in the incoming JSON payload. If valid, it iterates through each sample containing accelerometer, gyroscope, and status values and securely inserts them into the vibration table using prepared statements, which protect against SQL injection. The API provides structured JSON responses to indicate the success or failure of the data insertion, aiding in real-time diagnostics and error handling. This architecture ensures efficient, secure, and scalable data transfer between the ESP32 and the server. The code is in the Appendix.

4.3.4 Application Dashboard

A web app dashboard was created to give users easy access to the system, allowing them to see the data being collected from the motor in real time. The web app was built using React, Tailwind CSS, Vite and other web development tools. Users need to authenticate to use the application. The user must create an account using email and password to gain access to the monitoring system. New users can sign up and create an account to gain access to the system. In the database, a user's table was created, which contains the user's details, including name, email, phone and password, as shown in Figure 4.8. The concept of hashing was implemented when storing passwords in the database for safety purposes.

Showing rows 0 - 1 (2 total, Query took 0.0006 seconds.)

SELECT * FROM `users`

☐ Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

	id	name	email	phone	password	preferences
☐ Edit Copy Delete	2	Tendai	tendaimachaya12@gmail.com	0500285848	\$2y\$10\$e3p.JZoiq3u97UH2po2aj./Q.m1jy/wORp16/cnEQKL...	NULL
☐ Edit Copy Delete	3	Nigel Takunda Kadonzvo	bossdharma1@gmail.com	0599679936	\$2y\$10\$.wJCI6/KvA9qs44KDfgHfe1rkVIRI4rvY/3zsGHUsK...	NULL

☐ Check all | With selected: Edit Copy Delete Export

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Query results operations

Print Copy to clipboard Export Display chart Create view

Figure 4. 8: Users table that stores the login details for the current and new users.

After logging in the first page is the Dashboard, where the user has an overview of the systems operational metrics. Figure 4.9 shows an interface with a summary of the readings form the different sensors in the system.

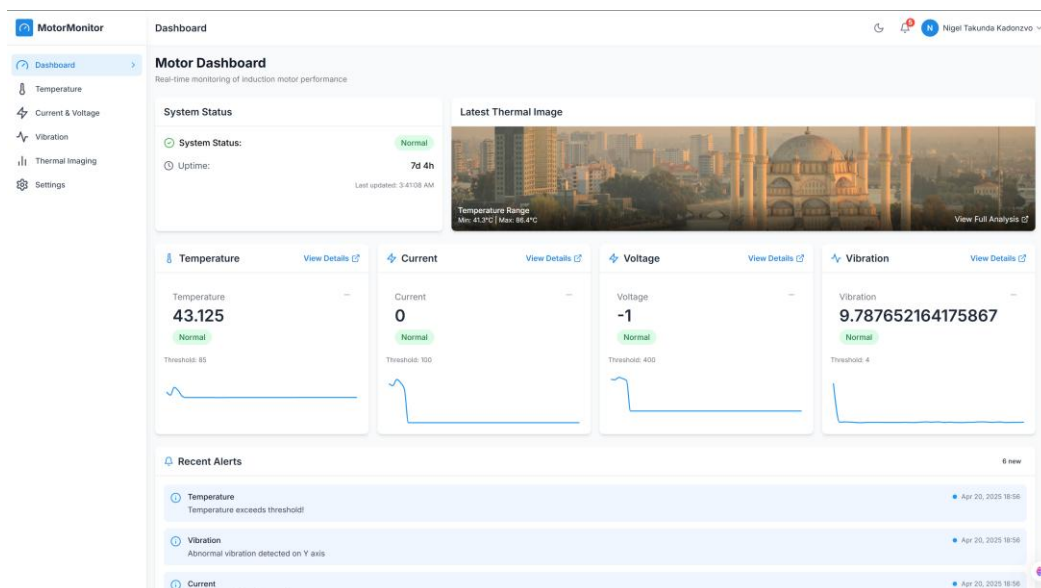


Figure 4. 9:Dashboard: Shows a summary of the data analysis.

The dashboard also has the Temperature analysis page, in Figure 4.10, which shows the current temperature on the surface of the motor. The page also allows the user to see the trends in the temperature that are being collected by the system over varying periods. There is an option to

view 1 1-hour, 6-hour or 1-day scale of the graph, which helps in giving a better understanding of how the temperature of the motor is behaving.

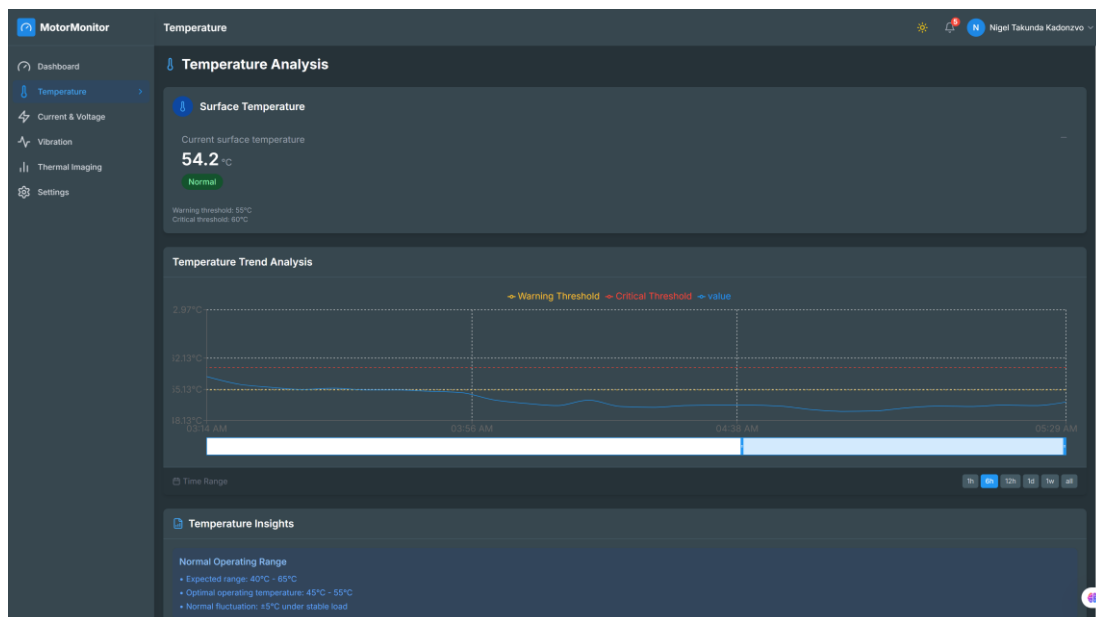


Figure 4. 10: Dashboard website: Surface temperature of the motor.

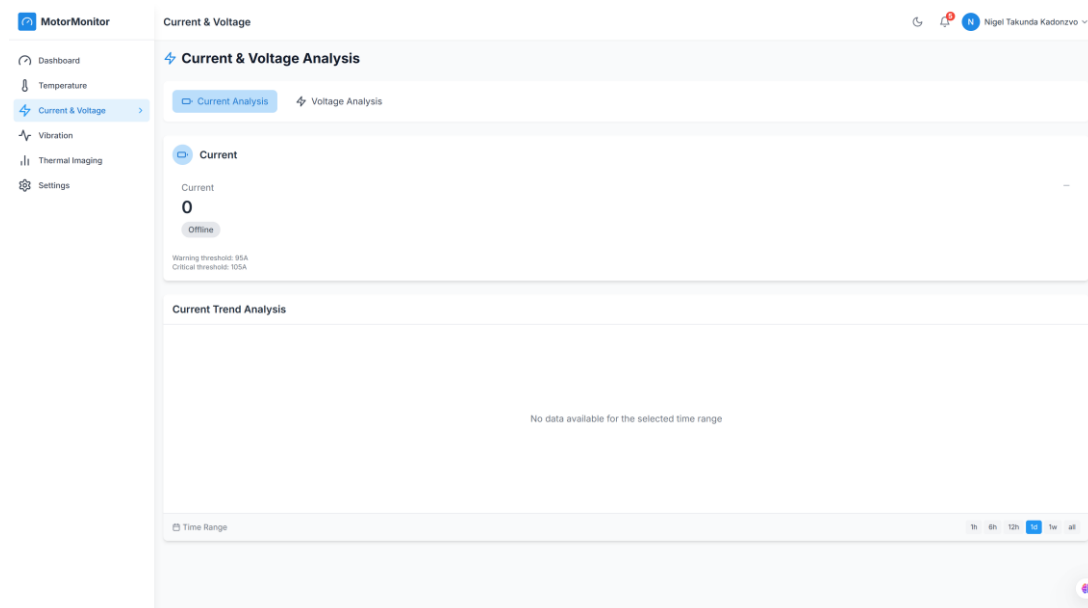


Figure 4. 11: Dashboard website: Current and Voltage of the motor.

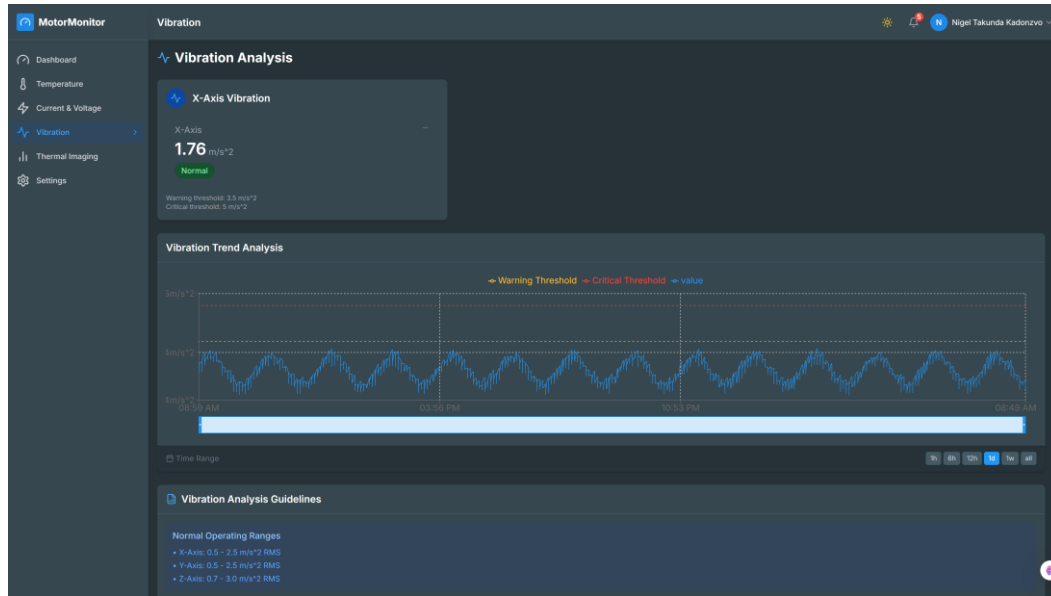


Figure 4. 12:Dashboard website: Vibration analysis page.

4.3.5 Machine Learning

Machine learning (ML) has become essential for predictive maintenance, especially in detecting anomalies in electrical motors. Motors are critical in many industries, and early fault detection can prevent costly downtimes and failures [6]. ML enables systems to automatically learn patterns from motor data, identifying deviations that signal potential issues. This allows for proactive maintenance, reducing disruptions [6].

The machine learning models implemented were in two parts. The first model was achieved from data collected on an available induction motor. That model was specifically for vibration since that was the only parameter that would be experimented with to introduce faults and get abnormal data. The second part was training a model using an online dataset from Kaggle, and this model was tested using that same online dataset, split into two groups: training and testing datasets.

4.3.5.1 Datasets Used

The data for training a machine learning model is important for the best-performing model. For this project, the parameters used to do predictive maintenance were quite difficult to introduce

faults or get anomalies from them. The induction motor used for this research could not be intentionally damaged or have any fault introduced to it, as it would damage it. The induction motor is reserved for teaching purposes. The only data that could be collected is the normal operational data. Also, if it were to wait until the failure happens to the motor, it takes an average of 25,000 hours for induction motors to have a fault, which was significantly beyond the timeline for this research [20].

The first dataset used for this research was the vibration data that was collected from the induction motor after causing artificial failures through supplying imbalanced power. Induction motors vibrate differently when they operate with a balanced power supply compared to having an imbalanced power supply. The induction motor produced a louder noise and made more vibrations when an imbalanced power supply was fed to it. Hence, a dataset was collected, with nearly 500,000 data values labelled as normal operations and 500,000 as abnormal/imbalanced. The data was collected at a sampling rate of 100 Hz for about an hour. Data preprocessing and training were done on the data using different analytical techniques to produce meaning from the data. So, for a realistic model, an online dataset from Kaggle was used to train a model that would identify faults for most of the parameters for this project.

The data for training a machine learning model is important for the best-performing model. For this project, the parameters used to do predictive maintenance were quite difficult to introduce faults or get anomalies from them. The induction motor used for this research could not be intentionally damaged or have any fault introduced to it, as it would damage it. The induction motor is reserved for teaching purposes. The only data that could be collected is the normal operational data. Also, if it were to wait until the failure happens to the motor, it takes an average of 25,000 hours for induction motors to have a fault, which was significantly beyond the timeline for this research [20].

The first dataset used for this research was the vibration data that was collected from the induction motor after causing artificial failures through supplying imbalanced power. Induction motors vibrate differently when they operate with a balanced power supply compared to having an imbalanced power supply. The induction motor produced a louder noise and made more vibrations when an imbalanced power supply was fed to it. Hence, a dataset was collected, with nearly 500,000 data values labelled as normal operations and 500,000 as abnormal/imbalanced. The data was collected at a sampling rate of 100 Hz for about an hour. Data preprocessing and training were done on the data using different analytical techniques to produce meaning from the data. So, for a realistic model, an online dataset from Kaggle was used to train a model that would identify faults for most of the parameters for this project.

4.3.5.2 Data Preprocessing

Data preprocessing was a key step in preparing the motor vibration data for machine learning models. The process began with data cleaning, where missing values in sensor readings were imputed using the mean of the respective features. Labels indicating fault status were encoded using LabelEncoder to convert them into numerical form [11]. The data was then divided into non-overlapping windows of 100 data points, allowing for better management and analysis. For each window, both time-domain features (mean, standard deviation, RMS, variance) and frequency-domain features (mean and standard deviation of the FFT) were extracted [13]. These features captured the statistical properties of the vibration signal and its frequency components.

Next, the features were scaled using MinMaxScaler to normalize them within the range [0, 1], ensuring all features were comparable for the machine learning models [11]. To evaluate model performance, the dataset was split into two: 80% for training and 20% for testing. To improve model generalization, data augmentation techniques, such as jittering, scaling, and rotation,

were applied [13]. Statistical methods were used to generate more data points within the range. The dataset ended up having 8 million data entries. This increased the diversity of the data and helped the models avoid overfitting.

4.3.5.3 Machine Learning Models

In this project, several machine learning models were evaluated to classify motor vibration states as normal or abnormal. The models used included Random Forest, k-Nearest Neighbors (k-NN), Decision Tree, and Artificial Neural Networks (ANN) [10].

Random Forest is a learning method that combines multiple decision trees to improve accuracy. It is particularly effective in handling large datasets and was used to assess its ability to classify motor vibration states. k-Nearest Neighbors (k-NN) is a simple, intuitive model that classifies data based on the majority class of its nearest neighbors. It was used to explore how well this model could handle the motor vibration data for classification tasks.

Decision Tree is a straightforward, interpretable model that splits data into branches based on feature values. It was employed to analyze the patterns in the motor vibration dataset and identify fault occurrences. Artificial Neural Networks (ANN) were the primary model used for fault detection due to their ability to learn complex, non-linear patterns in large, high-dimensional datasets [21]. ANNs automatically extract features from raw data, making it well-suited for time-series and frequency domain motor vibration data. Each model was tested to evaluate its effectiveness in predicting motor anomalies, with ANN being the most successful in capturing the complex patterns necessary for accurate fault detection [10].

CHAPTER 5: RESULTS AND DISCUSSION

5.1 System Performance Overview

The system's performance was evaluated with a focus on the hardware functionality, anomaly detection s, Arduino code, communication between the ESP32 and Raspberry Pi 5 via MQTT, and the interaction between the database, web app, and alerting mechanisms. This section outlines how each component worked together to ensure the system effectively monitored the induction motor in real-time.

5.2 ESP32, Sensors, Communication, Database and Raspberry Pi

One of the primary goals of this project was to develop a system capable of collecting data from an operating induction motor over an extended period using various sensors. This data would then be accumulated and used to train machine learning models once sufficient information was gathered. The system successfully collected sensor data and stored it in a database. It also established smooth wireless communication between the ESP32 and Raspberry Pi via MQTT, enabling real-time transfer of accelerometer data for edge processing. Powered by an external battery supply, the system demonstrated excellent energy efficiency. Additionally, the system facilitated seamless data transfer from the database to the web app, allowing real-time visualization of motor performance. Furthermore, it included an alerting mechanism that notified users whenever abnormal vibration data was detected, ensuring proactive maintenance.

5.3 Anomaly Detection

5.3.1 Running the models on PC

As mentioned in section 4.2.5.3, multiple machine learning models were used to train the vibration dataset. The dataset that was used is the one that was collected in the lab not the online dataset. These models were tested to classify motor vibration states as normal or abnormal. The

models evaluated included Random Forest, k-Nearest Neighbors, Decision Tree, and Artificial Neural Networks [10]. The models were high accuracy, that is, the patterns in vibration data are clear and distinct, where faults are manifested by measurable changes in frequency, amplitude, and other characteristics. Hence, effective feature engineering that included the extraction of time-domain and frequency-domain features ensured a wide array of inputs, thereby increasing the chances of the models to detect anomalies. Data normalization using techniques like MinMaxScaler ensured that all features were comparable, making it easier for the models to learn and classify the data accurately.

The Random Forest model achieved an impressive accuracy of 98.3%, indicating excellent performance in classifying motor vibration data (see Figure 5.1). Both precision and recall for the imbalanced (faulty) and normal classes were 98%, showing that the model effectively distinguished between normal and faulty states. The inference time was 0.0021 milliseconds. The confusion matrix highlights the model's minimal errors, with only 130 false positives (normal predicted as faulty) and 172 false negatives (faulty predicted as normal).

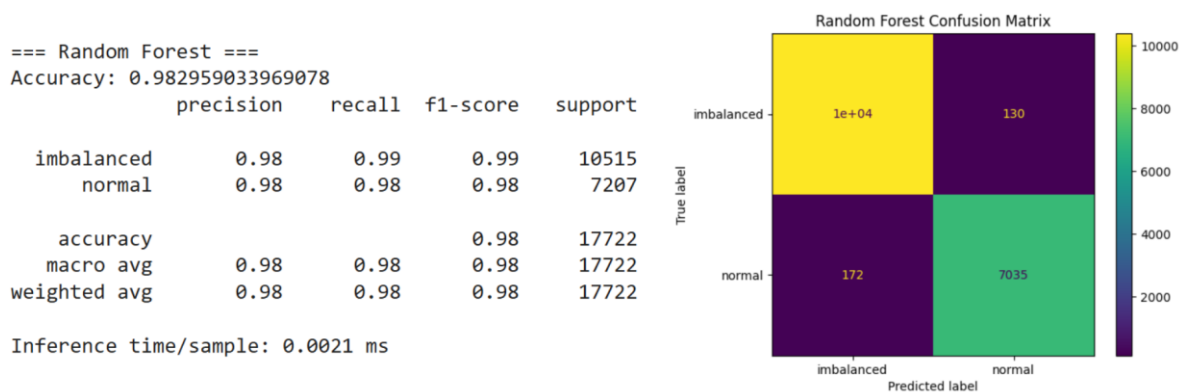


Figure 5. 1: Classification table and confusion matrix for the Random Forest model.

The k-Nearest Neighbors (k-NN) model achieved an accuracy of 98.17%, demonstrating strong performance in classifying motor vibration data (see Figure 5.2). The precision and recall for both imbalanced (faulty) and normal classes were 98%, indicating the model was highly effective in identifying both classes correctly. The confusion matrix shows minimal

misclassification, with 156 false positives (normal predicted as faulty) and 169 false negatives (faulty predicted as normal). The inference time was 1.3627 milliseconds.

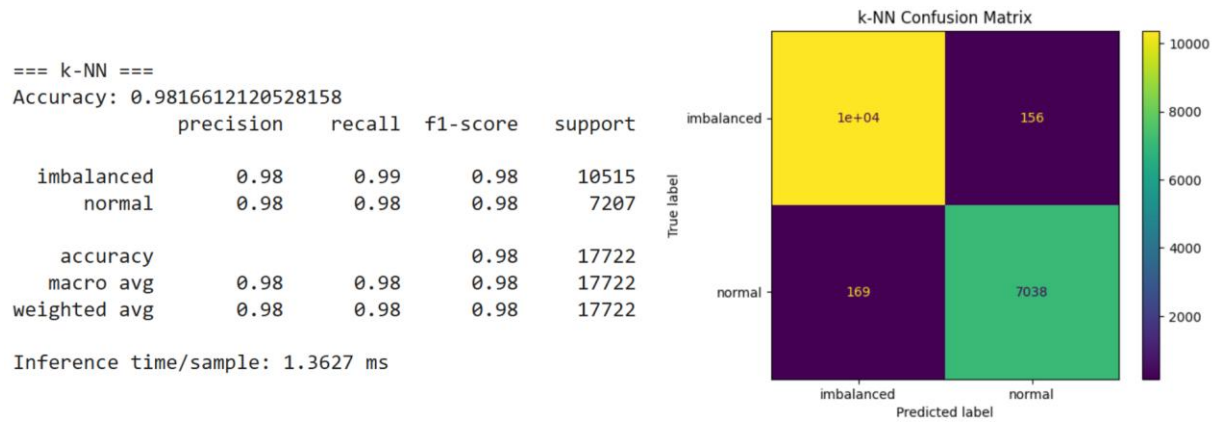


Figure 5. 2: Classification table and confusion matrix for the k-NN model.

The Decision Tree model achieved an accuracy of 96.7%, indicating solid performance in classifying motor vibration data (see Figure 5.3). The precision and recall for both imbalanced (faulty) and normal classes were 97% and 96%, respectively, showing that the model identified both classes effectively. The confusion matrix reveals minor misclassifications, with 303 false positives (normal predicted as faulty) and 289 false negatives (faulty predicted as normal). The model performed well. It demonstrated its effectiveness in detecting motor anomalies with high classification accuracy. The inference time was 0.0003 milliseconds.

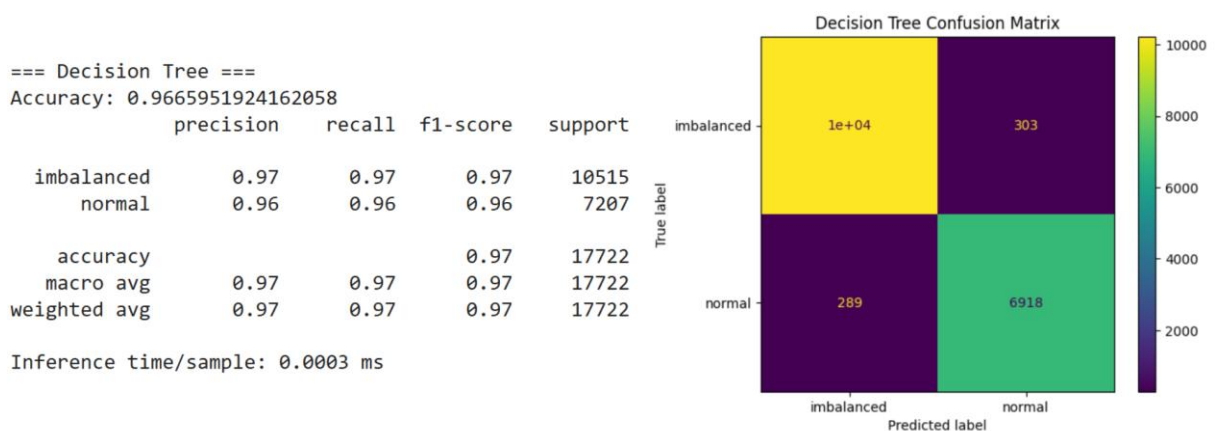


Figure 5.3: Classification table and confusion matrix for the Decision Tree model.

The Artificial Neural Network (ANN) model achieved an accuracy of 97.12%, and thus presents strong characteristics in classifying motor vibration data. The precision for the imbalanced (faulty) class was 98%, and for the normal class, it was 95%. Showing that the model was particularly accurate in detecting faulty states. The recall for the imbalanced class was 97%, while for the normal class it was 98%, indicating that the model effectively identified both faulty and non-faulty states. The confusion matrix reveals 354 false positives (normal predicted as faulty) and 156 false negatives (faulty predicted as normal), which are relatively few. The inference time was 0.0590 milliseconds.

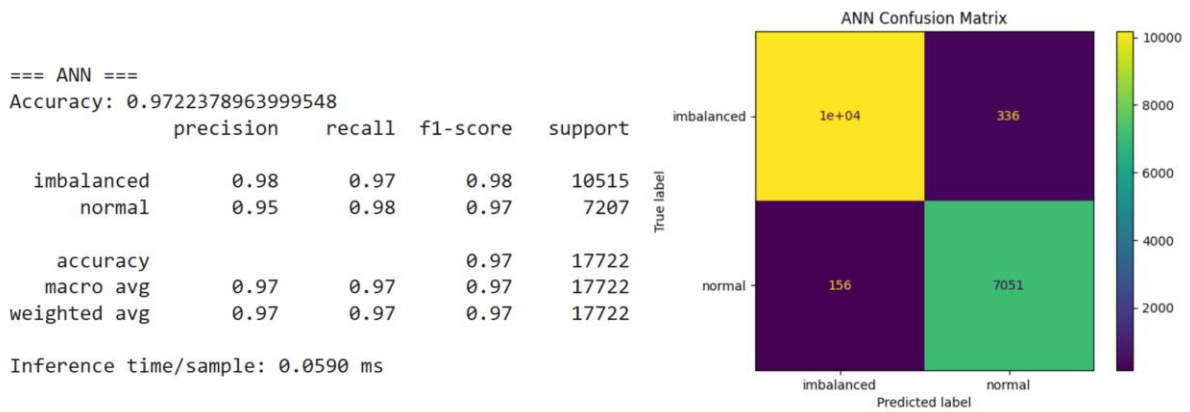


Figure 5.4: Classification table and confusion matrix for the ANN model.

5.3.2 Inference on the Raspberry Pi

There were some differences in inference time on the Raspberry Pi 5 compared to standard PC inference time after deploying the models. Inference times were higher, as expected, on the Raspberry Pi, considering that it has limited processing power, memory, and energy constraints. From the figure, Random Forest had an inference time of 0.00119 milliseconds on the Raspberry Pi, which was quite similar to that of the inference time on a PC. The accuracy of the model on Raspberry Pi was 99.6%, which is an advancement when compared to the results before its deployment in Raspberry Pi. This increase in accuracy can be attributed to overfitting,

where the model's performance became overly specialized to the training data, leading to enhanced performance on the test set.

```
=== Random Forest ===
Accuracy: 0.9962
```

	precision	recall	f1-score	support
imbalanced	1.00	1.00	1.00	2022
normal	1.00	0.99	1.00	1386
accuracy			1.00	3408
macro avg	1.00	1.00	1.00	3408
weighted avg	1.00	1.00	1.00	3408

```
Inference time/sample: 0.00119 ms
```

Figure 5.5: Classification table for the Random Forest model running on Raspberry Pi.

In Figure 5.6, the k-NN model showed an inference time of 2.90360 milliseconds on the Raspberry Pi. The inference time was significantly higher than the 1.3627 milliseconds achieved when running on the PC. The model's accuracy increased to 99.9% when deployed on the Raspberry Pi, contrary to expectations. It was anticipated that the accuracy would decrease due to the limited computational resources of the Raspberry Pi, but instead, the model showed an unexpected improvement in performance.

```
=== k-NN ===
Accuracy: 0.9997
```

	precision	recall	f1-score	support
imbalanced	1.00	1.00	1.00	2022
normal	1.00	1.00	1.00	1386
accuracy			1.00	3408
macro avg	1.00	1.00	1.00	3408
weighted avg	1.00	1.00	1.00	3408

```
Inference time/sample: 2.90360 ms
```

Figure 5.6: Classification table for the k-NN model running on Raspberry Pi.

In Figure 5.7, the Decision Tree model demonstrated an inference time of 0.00022 milliseconds on the Raspberry Pi, which is highly efficient given the constraints of the device. The model's

accuracy dropped to 91.7% when running on the Raspberry Pi, which aligns with expectations. The decrease in accuracy is due to the limitations in computational resources, which can impact the model's ability to make accurate predictions.

```

=== Decision Tree ===
Accuracy: 0.9170

```

	precision	recall	f1-score	support
imbalanced	0.90	0.96	0.93	2022
normal	0.94	0.85	0.89	1386
accuracy			0.92	3408
macro avg	0.92	0.91	0.91	3408
weighted avg	0.92	0.92	0.92	3408

```

Inference time/sample: 0.00022 ms

```

Figure 5.7: Classification table for the Decision Tree model running on Raspberry Pi.

While running the ANN model on a Raspberry Pi, as shown in Figure 5.8, its inference time was 0.00606 milliseconds compared to the time when running on a PC, which was longer on the Raspberry Pi. Also, the model's accuracy dropped to 94.5% on the Raspberry Pi, which is expected. Such an increase in inference time and drop in accuracy is attributed to the constrained computational capabilities of the Raspberry Pi affecting the performance and efficiency of the model on deployment in a resource-constrained device.

```

=== ANN (TFLite) ===
Accuracy: 0.9454

```

	precision	recall	f1-score	support
imbalanced	1.00	0.91	0.95	2022
normal	0.88	1.00	0.94	1386
accuracy			0.95	3408
macro avg	0.94	0.95	0.94	3408
weighted avg	0.95	0.95	0.95	3408

```

Inference time/sample: 0.00606 ms
(venv) Nijo@Nijo:~/Desktop/capstone $ |

```

Figure 5.8: Classification table for the ANN model running on Raspberry Pi.

5.3.3 Selection of Machine Learning Model

The ANN model was selected for this project based on the results and performance of the trained models. The other models were not picked for many reasons. The Random Forest model was not picked because of its high accuracy, which could potentially overfit and reduce the generalization of unseen data. The k-NN model was dropped because of its very high inference on the Raspberry Pi and the unexpected increase in accuracy, which may indicate overfitting. The Decision Tree model performed well, but was dropped because of the ANN model.

The ANN model was picked because, from the literature review, ANN models perform well when doing predictive maintenance [13]. ANN model is robust and capable of learning complex patterns, including nonlinear patterns in high-dimensional datasets. Motor vibration data often exhibits intricate relationships that other models do not easily capture. ANN's deep learning capabilities allow it to extract meaningful features from the raw data automatically [13].

CHAPTER 6: CONCLUSION AND FUTURE WORKS

6.1 Conclusion

This project successfully developed and implemented an IoT-based anomaly detection system for rotating electrical machinery, specifically focusing on 3-phase induction motors. Integrating Internet of Things (IoT) technology with embedded machine learning enabled real-time monitoring and fault detection, addressing the limitations of traditional reactive and preventive maintenance strategies. The system provided real-time insights into the motor's operational health by using multiple sensors to collect multi-modal data (vibration, temperature and current).

The system demonstrated its capability to collect and process data over extended periods, even with simulated faults, enabling it to detect anomalies such as abnormal vibrations caused by unbalanced power supplies. The use of MQTT for wireless communication between the ESP32 and Raspberry Pi 5 ensured smooth, low-latency data transfer for edge processing. A web-based dashboard enabled real-time visualization, enhancing user interaction with the system. The implementation of machine learning algorithms, such as Artificial Neural Networks (ANN), achieved high classification accuracy (94%) running on the Raspberry Pi, showing that the system could effectively predict and alert users about potential failures.

This research highlights the effectiveness of IoT-based predictive maintenance systems, which can operate independently without relying heavily on cloud infrastructure. The system successfully minimized downtime and reduced maintenance costs.

6.2 Future Works

While this project successfully demonstrated the feasibility of an IoT-based anomaly detection system for rotating electrical machines, several areas can be explored further to enhance the system's performance and applicability.

- i. The system primarily focused on vibration data to detect faults caused by unbalanced power supplies. Future work can involve incorporating additional fault types, such as electrical imbalances, bearing wear, or rotor faults, to expand the system's capability in detecting a wider range of anomalies.
- ii. Although simulated faults were introduced to generate data for training machine learning models, obtaining a more extensive and diverse dataset from real-world motor failures would improve model accuracy. Collaboration with industries or access to large-scale failure data would allow the system to generalize better across different motor types and conditions.
- iii. Selection of a well-functioning thermal imaging sensor. The one used in this project had issues with its I2C. There were constant errors when the data was being read.

REFERENCES

- [1] P. Gackowiec, “General overview of maintenance strategies – concepts and approaches,” *Multidisciplinary Aspects of Production Engineering*, vol. 2, no. 1, pp. 126–139, Sep. 2019, doi: 10.2478/mape-2019-0013.
- [2] J. Narkarunai Arasu Malaiyappan, G. Krishnamoorthy, and S. Jangoan, “Predictive Maintenance using Machine Learning in Industrial IoT,” *International Journal of Innovative Science and Research Technology (IJISRT)*, pp. 1909–1915, Apr. 2024, doi: 10.38124/ijisrt/IJISRT24MAR984.
- [3] A. Benhanifia, Z. Ben Cheikh, P. M. Oliveira, A. Valente, and J. Lima, “Systematic review of predictive maintenance practices in the manufacturing sector,” Jun. 01, 2025, *Elsevier B.V.* doi: 10.1016/j.iswa.2025.200501.
- [4] W. Torell, S. Electric, and V. Avelar, “Mean Time Between Failure: Explanation and Standards Mean Time Between Failure: Explanation and Standards Revision 1.” [Online]. Available: <https://www.researchgate.net/publication/251895269>
- [5] P. J. Tavner, “Review of condition monitoring of rotating electrical machines,” *IET Electr Power Appl*, vol. 2, no. 4, pp. 215–247, 2008, doi: 10.1049/IET-EPA:20070280.
- [6] O. Das, D. Bagci Das, and D. Birant, “Machine learning for fault analysis in rotating machinery: A comprehensive review,” Jun. 01, 2023, *Elsevier Ltd.* doi: 10.1016/j.heliyon.2023.e17584.
- [7] S. D. P. Ragavendiran, D. Shahakar, D. S. Kumari, A. S. Yadav, P. M. Arthi, and N. Rajesha, “Predictive Maintenance of Electrical Machines using Machine Learning and Condition Monitoring Data,” in *Proceedings - 3rd International Conference on Advances in Computing, Communication and Applied Informatics, ACCAI 2024*, Institute of Electrical and Electronics Engineers Inc., 2024. doi: 10.1109/ACCAI61061.2024.10601981.
- [8] P. Tavner, L. Ran, J. Penman, and H. Sedding, “Condition monitoring of rotating electrical machines,” *Condition Monitoring of Rotating Electrical Machines*, pp. 1–250, Jan. 2008, doi: 10.1049/PBPO056E.
- [9] T. Vaimann and A. Kallaste, “Condition Monitoring of Electrical Machines,” 2012. [Online]. Available: <https://www.researchgate.net/publication/262674501>
- [10] I. Ullah, N. Khan, S. A. Memon, W. G. Kim, J. Saleem, and S. Manzoor, “Vibration-Based Anomaly Detection for Induction Motors Using Machine Learning,” *Sensors*, vol. 25, no. 3, Feb. 2025, doi: 10.3390/s25030773.
- [11] A. Lazcano and M. A. Jaramillo-Morán, “Data preprocessing techniques and neural networks for trended time series forecasting,” *Appl Soft Comput*, vol. 174, p. 113063, Apr. 2025, doi: 10.1016/J.ASOC.2025.113063.
- [12] F. Derbel and F. Strakosch, “Integrated Sensor based Smart Diagnostic and Online Monitoring of Industrial Systems,” in *Proceedings - 2022 4th International Conference*

- on *Applied Automation and Industrial Diagnostics, ICAAID 2022*, Institute of Electrical and Electronics Engineers Inc., 2022. doi: 10.1109/ICAAID51067.2022.9799489.
- [13] O. Mey, W. Neudeck, A. Schneider, and O. Enge-Rosenblatt, “Machine Learning-Based Unbalance Detection of a Rotating Shaft Using Vibration Data.”
 - [14] P. P. Ray, “A review on TinyML: State-of-the-art and prospects,” Apr. 01, 2022, *King Saud bin Abdulaziz University*. doi: 10.1016/j.jksuci.2021.11.019.
 - [15] H. Nizam, “Tiny Machine Learning (TinyML) Systems.” [Online]. Available: <https://www.researchgate.net/publication/386579238>
 - [16] R. David *et al.*, “TENSORFLOW LITE MICRO: EMBEDDED MACHINE LEARNING ON TINYML SYSTEMS,” 2021.
 - [17] S. Hymel *et al.*, “Edge Impulse: An MLOps Platform for Tiny Machine Learning,” Nov. 2022, [Online]. Available: <http://arxiv.org/abs/2212.03332>
 - [18] A. K. A. Aboagye-Otchere and A. K. Bleboo, “Condition Monitoring of Induction Motors Using IoT,” 2024.
 - [19] A. Kucuker and M. Bayrak, “Detection of mechanical imbalances of induction motors with instantaneous power signature analysis,” *Journal of Electrical Engineering and Technology*, vol. 8, no. 5, pp. 1116–1121, 2013, doi: 10.5370/JEET.2013.8.5.1116.
 - [20] P. Tavner, L. Ran, J. Penman, and H. Sedding, *Condition monitoring of rotating electrical machines*. Institution of Engineering and Technology, 2008. doi: 10.1049/PBPO056E.
 - [21] L. A. E. Noussaiba and F. Abdelaziz, “ANN-based fault diagnosis of induction motor under stator inter-turn short-circuits and unbalanced supply voltage,” *ISA Trans*, vol. 145, pp. 373–386, Feb. 2024, doi: 10.1016/J.ISATRA.2023.11.020.

APPENDIX

